

SELENIUM AUTOMATION ASSIGNMENT

Assignment 3 – Dynamic Dropdowns & Checkboxes

Execution Report • Selenium with Python • Selenium Web Form

Item	Details
Assignment	3 – Dynamic Dropdowns & Checkboxes
Application Under Test	Selenium Web Form
Automation Tool	Selenium WebDriver
Programming Language	Python
Browser	Google Chrome
Checkboxes	Checked checkbox + Default checkbox
Autocomplete	HTML datalist / search-as-you-type field
Matching Option	San Francisco
Execution Status	PASSED

This report documents the implementation and successful execution of the checkbox and dynamic datalist automation task, including locator strategies, validation logic, source code, and execution evidence.

1. Objective

The objective of this assignment is to automate a web form containing multiple checkboxes and a search-as-you-type datalist. The test demonstrates checkbox selection and state verification using `is_selected()`, followed by dynamic option handling through a loop.

The matching autocomplete option is identified from the available datalist options and entered into the input field. Assertions are then used to verify both checkbox states and the selected autocomplete value.

2. Assignment Requirements

No.	Requirement	Implementation
1	Open form	Selenium Web Form
2	Select specific checkboxes	Default checkbox selected; checked checkbox verified
3	Verify checkbox state	<code>.is_selected()</code>
4	Type into autocomplete	Type into input connected to the datalist
5	Loop through suggestions	Loop through <code>#my-options</code> option elements
6	Select matching option	San Francisco
7	Validate result	Assertions + terminal output

3. Test Environment

Component	Configuration
Operating System	Windows
IDE	Visual Studio Code
Language	Python 3.14 (64-bit)
Automation Framework	Selenium WebDriver
Browser	Google Chrome
Test Page	Selenium Web Form

4. Implementation Approach

Step 1: Launch Chrome using Selenium WebDriver.

Step 2: Navigate to the Selenium Web Form.

Step 3: Locate the Default checkbox and select it when necessary.

Step 4: Use `is_selected()` to verify the Default checkbox state.

Step 5: Locate the already checked checkbox and verify its selected state.

Step 6: Locate the autocomplete input through its `list='my-options'` relationship.

Step 7: Type into the autocomplete input.

Step 8: Retrieve the datalist option elements and loop through them.

Step 9: Find the matching 'San Francisco' option and enter it into the input.

Step 10: Validate both checkbox states and the autocomplete value with assertions.

5. Locator Strategy

Element	Locator	Purpose
Default checkbox	By.ID, my-check-2	Select and verify checkbox
Checked checkbox	By.ID, my-check-1	Verify existing selection
Autocomplete input	CSS: input[list='my-options']	Type search value
Datalist options	CSS: #my-options option	Retrieve and loop through options

6. Key Automation Logic

```
# Verify checkbox state
assert default_checkbox.is_selected()

# Locate the datalist input
autocomplete = wait.until(
    EC.element_to_be_clickable(
        (By.CSS_SELECTOR, "input[list='my-options']")
    )
)

# Loop through datalist options
suggestions = driver.find_elements(
    By.CSS_SELECTOR, "#my-options option"
)

for suggestion in suggestions:
    option_value = suggestion.get_attribute("value")

    if option_value.lower() == "san francisco":
        autocomplete.clear()
        autocomplete.send_keys(option_value)
        break

# Final validation
assert autocomplete.get_attribute("value") == "San Francisco"
```

7. Complete Test Script

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

driver = webdriver.Chrome()
driver.get("https://www.selenium.dev/selenium/web/web-form.html")
driver.maximize_window()

wait = WebDriverWait(driver, 10)

# Select and verify Default checkbox
default_checkbox = wait.until(
    EC.presence_of_element_located((By.ID, "my-check-2"))
)

if not default_checkbox.is_selected():
    default_checkbox.click()

assert default_checkbox.is_selected()
print("Default checkbox selected:", default_checkbox.is_selected())

# Verify already checked checkbox
checked_checkbox = driver.find_element(By.ID, "my-check-1")
assert checked_checkbox.is_selected()
print("Checked checkbox selected:", checked_checkbox.is_selected())

# Locate autocomplete input
autocomplete = wait.until(
    EC.element_to_be_clickable(
        (By.CSS_SELECTOR, "input[list='my-options']")
    )
)

# Type into the autocomplete field
autocomplete.send_keys("S")

# Retrieve and loop through datalist options
suggestions = driver.find_elements(
    By.CSS_SELECTOR, "#my-options option"
)

selected_option = None

for suggestion in suggestions:
    option_value = suggestion.get_attribute("value")
    print("Suggestion:", option_value)

    if option_value.lower() == "san francisco":
        selected_option = option_value
        autocomplete.clear()
        autocomplete.send_keys(option_value)
        print("Matching option selected:", option_value)
        break

# Validation
assert selected_option == "San Francisco"
assert default_checkbox.is_selected()
assert checked_checkbox.is_selected()
assert autocomplete.get_attribute("value") == "San Francisco"

print("Checkbox validation: PASSED")
print("Autocomplete validation: PASSED")
print("Assignment 3 Passed!")

driver.quit()
```

8. Execution Evidence

The submitted execution screenshots were reviewed. The VS Code terminal shows successful checkbox verification, the matching autocomplete option, both validation messages, and the final 'Assignment 3 Passed!' message. The browser screenshot shows the Selenium Web Form with both checkboxes selected during the execution.

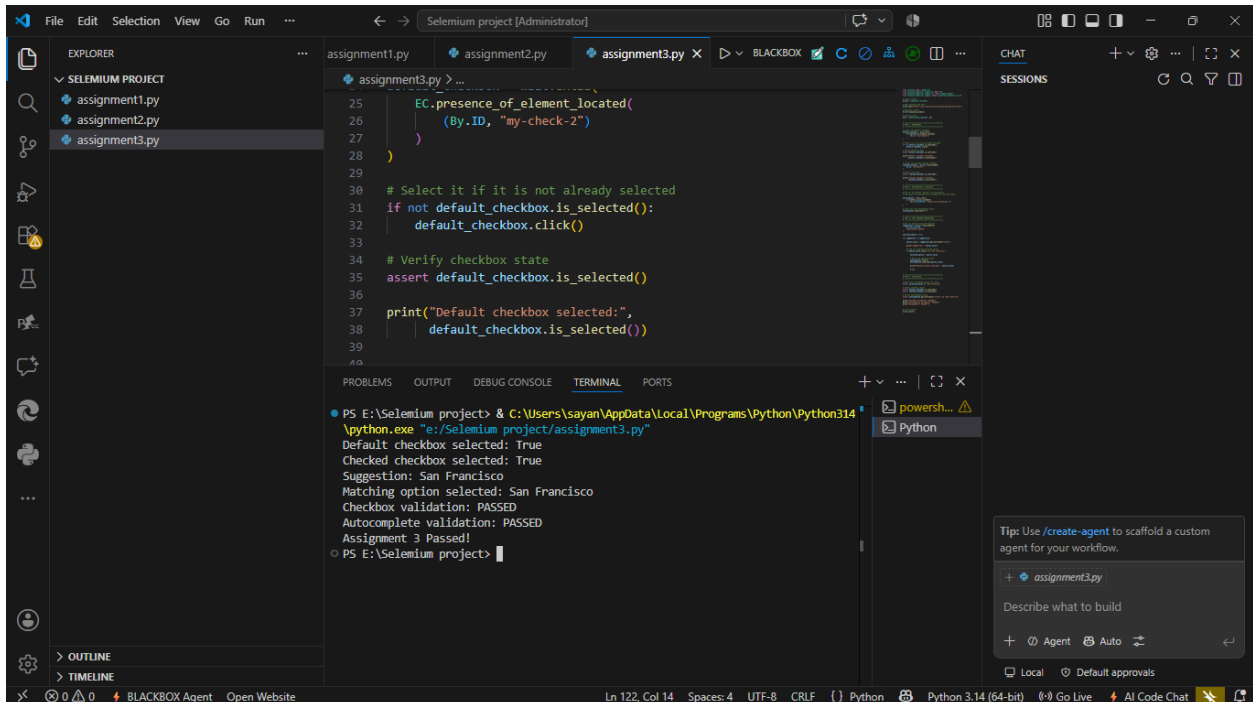


Figure 1 – VS Code terminal showing successful Assignment 3 execution and validation.

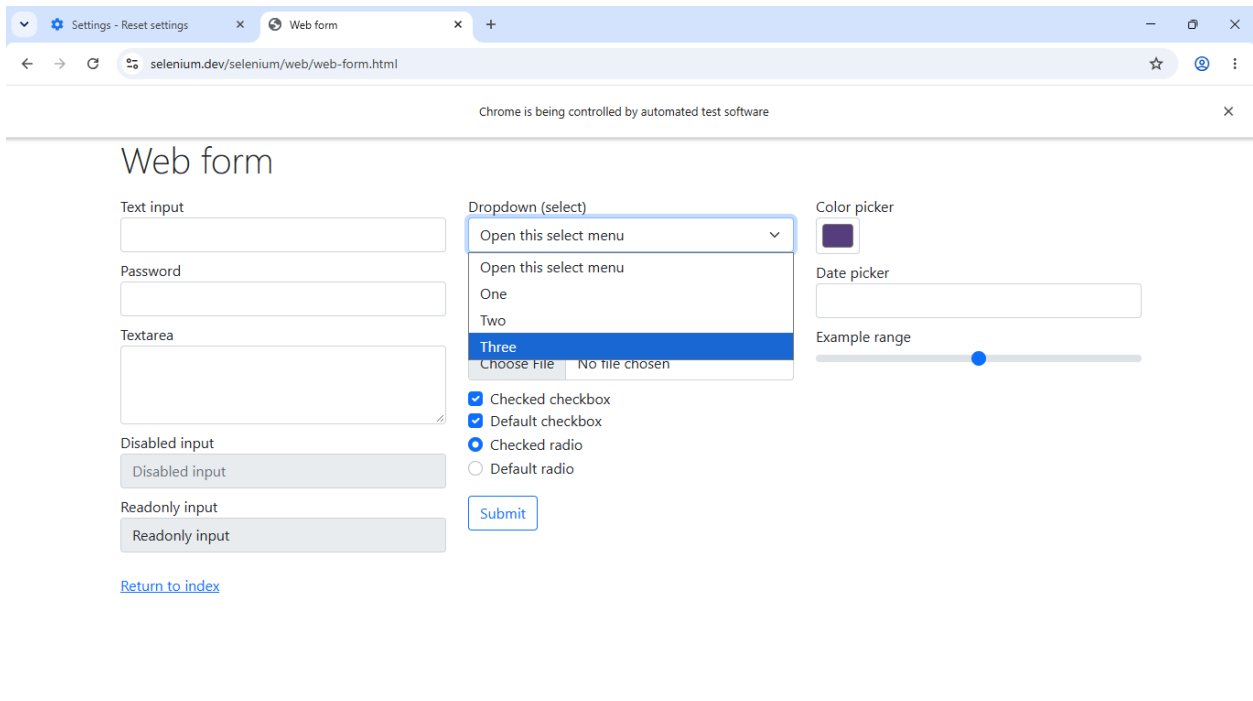


Figure 2 – Selenium Web Form showing the checkbox state during execution.

9. Test Result

Validation Point	Expected	Observed	Status
Default checkbox	Selected	True	PASS
Checked checkbox	Selected	True	PASS
Autocomplete input	Accept typed value	Accepted	PASS
Suggestion loop	Find matching option	San Francisco found	PASS
Checkbox validation	Both states true	PASSED	PASS
Autocomplete validation	Value = San Francisco	PASSED	PASS
Overall execution	All checks pass	Assignment 3 Passed!	PASS

10. Conclusion

The Dynamic Dropdowns & Checkboxes assignment was successfully completed using Selenium WebDriver with Python. The test selected and verified checkbox states using `is_selected()`, interacted with the search-as-you-type datalist input, looped through the available option elements, and selected the required matching option, San Francisco.

The execution evidence confirms that the checkbox and autocomplete validations passed and the terminal displayed 'Assignment 3 Passed!'.

Final Status: PASSED

End of Report